

lab3：二叉树操作

实验目的

1. 掌握二叉树的基本结构和操作
2. 实现二叉树的层次计算和节点遍历
3. 理解广度优先搜索（BFS）在二叉树中的应用

实验内容

1. 数据结构定义

链表二叉树中每个节点存储一个字符，结构定义（供参考，也可以查看chapter5.h文件）如下：

```
1  typedef struct binTreeNode {
2      char data;                // 节点存储的字符
3      struct binTreeNode *left; // 左子树指针
4      struct binTreeNode *right; // 右子树指针
5  } BinTreeNode;
6
7  typedef struct {
8      BinTreeNode *root;        // 二叉树根节点
9  } BinTree;
```

2. 实验要求

(1) 计算二叉树的层数

实现函数计算二叉树的深度（层数）：

```
int getTreeDepth(struct BinTreeNode *);
```

(2) 列出特定层级的节点元素

定义函数listLayerNode(int)，列出二叉树中某层的所有结点

(3) 按广度优先顺序列出节点

按照广度优先周游（层次遍历）的顺序列出第（2）小题要求的节点元素。

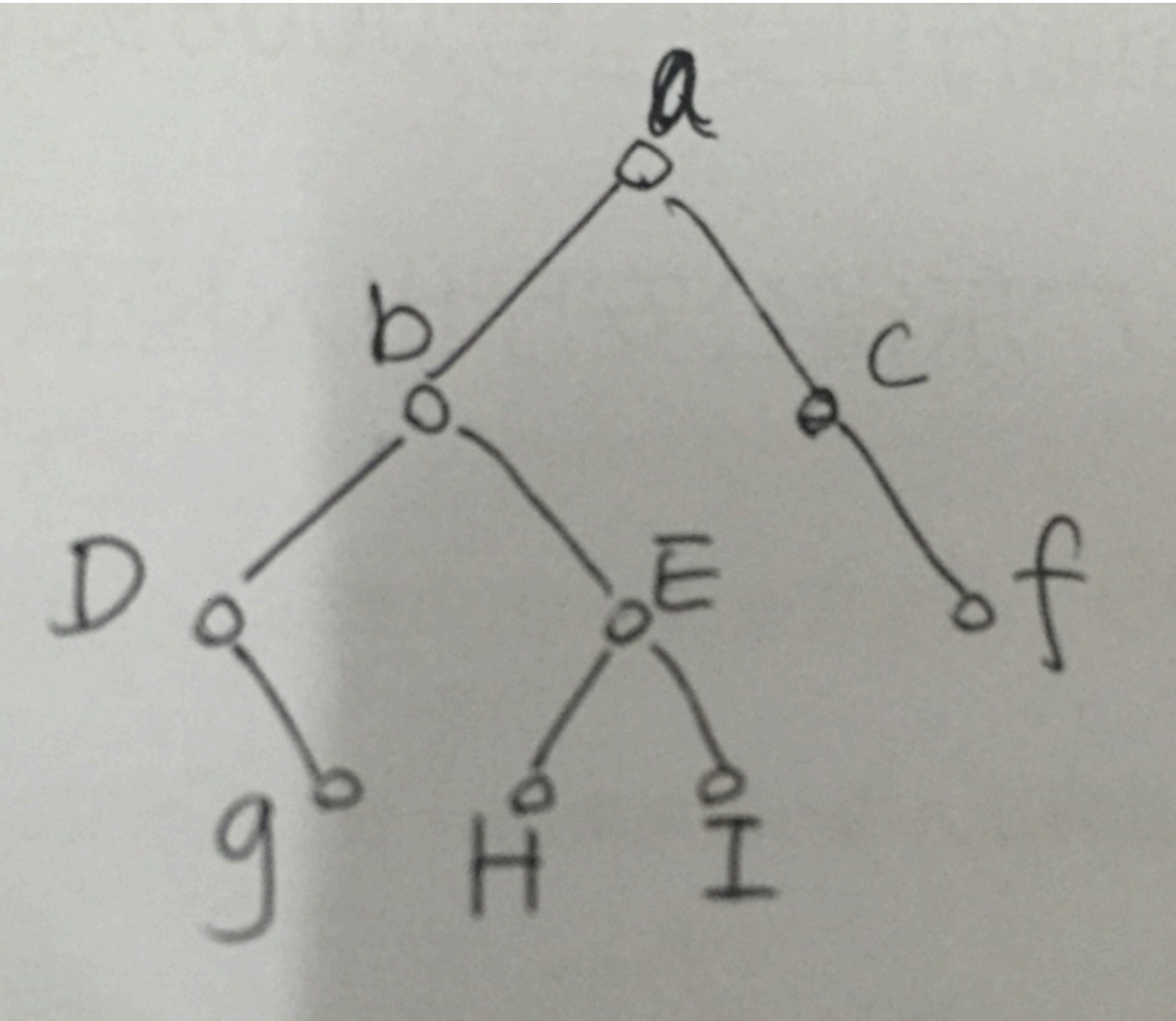
3. 实验提示

- 基于提供的 bintr.c 程序进行修改

- ## 实验步骤

abD.g..EH..I..c.f.. (注意f后面有两".")

对应二叉树结构，如下图：



步骤3：功能测试

程序提供多个测试选项（输入2-3-4-5），用于验证不同功能：

- 选项1：创建新二叉树
- 选项2-5：执行不同的遍历和计算操作

实验预期结果

预期输出

1. 二叉树层数：4层
2. 最后一层节点：G, H, I
3. 倒数第二层节点：D, E, F
4. 广度优先顺序：A, B, C, D, E, F, G, H, I

实现建议

1. **使用队列实现BFS**：需要实现队列数据结构辅助层次遍历
2. **递归计算深度**：使用后序遍历计算二叉树深度
3. **层级节点收集**：使用递归或迭代方式获取特定层级节点
4. **内存管理**：注意动态内存的分配和释放

思考题

1. 如何优化算法以减少时间复杂度？
2. 如果二叉树非常大，哪种实现方式更节省内存？
3. 如何扩展程序以支持其他类型的二叉树遍历？

参考资料

- `chapter5.h`：二叉树函数库头文件
- `chapter5.c`：二叉树函数库实现
- `bintr.c`：主程序文件

通过本实验，学生将深入理解二叉树的结构特性、遍历算法和实际应用，为后续学习更复杂的数据结构打下坚实基础。